

React JS Hands-On Labs Manual

Complete Reference Guide & Full Source Code Implementation (Labs 1 – 8)

LABORATORY EXERCISE INDEX

Lab 1: TrainersApp (React Router v6)

Lab 2: ShoppingApp (Props & Components)

Lab 3: CounterApp (React State)

Lab 4: BloggerApp (Conditional Rendering)

Lab 5: EmployeeApp (Context API)

Lab 6: TicketRaisingApp (Forms)

Lab 7: MailRegisterApp (Validation)

Lab 8: FetchUserApp (REST API Integration)

Lab 1: React Router Integration (TrainersApp)

Estimated Time: 60 Minutes | Application: TrainersApp | Focus: Routing & URL Parameters

1. OBJECTIVES & THEORETICAL CONCEPTS

- **Need & Benefits of React Router:** React is a Single Page Application (SPA) library. React Router provides client-side routing, enabling navigation without triggering full page reloads, preserving state and optimizing performance.
- **Core Components:**
 - **BrowserRouter:** Uses HTML5 history API to sync UI with URL.
 - **Routes:** Container component for defining route mappings (v6 requirement).
 - **Route:** Defines path mapping to specific component element.
 - **Link:** Provides accessible navigation links instead of standard HTML <a> tags.
- **URL Parameter Passing:** Access dynamic route parameters using the `useParams()` hook from `react-router-dom`.

2. INSTALLATION & SETUP COMMAND

Terminal / Command Prompt

```
# Create React App and install react-router-dom v6
npx create-react-app trainersapp
cd trainersapp
npm install react-router-dom@6
```

3. SOURCE CODE IMPLEMENTATION

src/trainer.js

```
class Trainer {
  constructor(trainerId, name, email, phone, technology, skills) {
    this.trainerId = trainerId;
    this.name = name;
    this.email = email;
```

```

    this.phone = phone;
    this.technology = technology;
    this.skills = skills;
  }
}

export default Trainer;

```

src/TrainersMock.js

```

import Trainer from "../trainer";

const trainersMock = [
  new Trainer(
    't-syed8',
    'Syed Khaleelullah',
    'khaleelullah@cognizant.com',
    '97676516962',
    '.NET',
    ['C#', 'SQL Server', 'React', '.NET Core']
  ),
  new Trainer(
    't-jojo',
    'Jojo Jose',
    'jojo@cognizant.com',
    '9897199231',
    'Java',
    ['Java', 'JSP', 'Angular', 'Spring']
  ),
  new Trainer(
    't-elisa',
    'Elisa Jones',
    'elisa@cognizant.com',
    '9871212235',
    'Python',
    ['Python', 'Django', 'Angular']
  )
];

export default trainersMock;

```

src/Home.js

```

import React from 'react';

function Home() {
  return (
    <div>
      <h2>Welcome to My Academy trainers page</h2>
    </div>
  );
}

export default Home;

```

src/TrainersList.js

```

import React from 'react';
import { Link } from 'react-router-dom';

```

```

function TrainersList({ trainers }) {
  return (
    <div>
      <h2>Trainers List</h2>
      <ul>
        {trainers.map((trainer) => (
          <li key={trainer.trainerId}>
            <Link to={`/${trainer.trainerId}`}>
              {trainer.name}
            </Link>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default TrainersList;

```

src/TrainerDetails.js

```

import React from 'react';
import { useParams } from 'react-router-dom';

function TrainerDetails({ trainers }) {
  const { id } = useParams();
  const trainer = trainers.find((t) => t.trainerId === id);

  if (!trainer) {
    return <h2>Trainer Not Found!</h2>;
  }

  return (
    <div>
      <h2>Trainers Details</h2>
      <h3>{trainer.name} ({trainer.technology})</h3>
      <p>{trainer.email}</p>
      <p>{trainer.phone}</p>
      <ul>
        {trainer.skills.map((skill, index) => (
          <li key={index}>{skill}</li>
        ))}
      </ul>
    </div>
  );
}

export default TrainerDetails;

```

src/App.js

```

import React from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
import trainersMock from './TrainersMock';
import Home from './Home';
import TrainersList from './TrainersList';
import TrainerDetails from './TrainerDetails';

function App() {
  return (
    <BrowserRouter>

```

```

<div style={{ padding: '15px' }}>
  <h1 style={{ fontSize: '32px', margin: '0 0 10px 0' }}>My Academy Trainers App</h1>
  <div style={{ fontSize: '18px', margin: '10px 0' }}>
    <Link to="/" style={{ color: 'purple' }}>Home</Link> | <Link to="/trainers"
style={{ color: 'purple' }}>Show Trainers</Link>
  </div>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/trainers" element={<TrainersList trainers={trainersMock} />} />
    <Route path="/trainer/:id" element={<TrainerDetails trainers={trainersMock} />} />
  </Routes>
</div>
</BrowserRouter>
);
}

export default App;

```

4. OUTPUT VISUALIZATION

RENDERED UI - TRAINER DETAILS ROUTE (/TRAINER/T-SYED8)

My Academy Trainers App

[Home](#) | [Show Trainers](#)

Trainers Details

Syed Khaleelullah (.NET)

khaleelullah@cognizant.com

97676516962

- C#
- SQL Server
- React
- .NET Core

Lab 2: Props & Component Rendering (shoppingapp)

Estimated Time: 60 Minutes | Application: shoppingapp | Focus: Component Props & List Rendering

1. OBJECTIVES & THEORETICAL CONCEPTS

- **Props (Properties):** Read-only inputs passed from parent components to child components to configure display and behavior.
- **Default Props:** Fallback values defined on components to handle scenarios where optional props are omitted by the caller.

- **State vs Props:** Props are immutable and passed down; State is mutable, managed internally within a component, and triggers re-renders on update.
- **ReactDOM.render() / createRoot():** Entry point method used to render React element trees into the real DOM container node.

2. SOURCE CODE IMPLEMENTATION

src/Cart.js

```
import React from 'react';

export class Cart extends React.Component {
  render() {
    return (
      <table border="1" cellPadding="8" cellSpacing="0" style={{ borderCollapse: 'collapse',
margin: '0 auto', color: 'green' }}>
        <thead>
          <tr>
            <th>Name</th>
            <th>Price</th>
          </tr>
        </thead>
        <tbody>
          {this.props.item.map((item, index) => {
            return (
              <tr key={index}>
                <td>{item.itemname}</td>
                <td>{item.price}</td>
              </tr>
            );
          })}
        </tbody>
      </table>
    );
  }
}

export default Cart;
```

src/OnlineShopping.js

```
import React from 'react';
import { Cart } from './Cart';

export class OnlineShopping extends React.Component {
  render() {
    const CartInfo = [
      { itemname: "Laptop", price: 80000 },
      { itemname: "TV", price: 120000 },
      { itemname: "Washing Machine", price: 50000 },
      { itemname: "Mobile", price: 30000 },
      { itemname: "Fridge", price: 70000 }
    ];

    return (
      <div className="mydiv" style={{ textAlign: 'center', marginTop: '20px' }}>
        <h1 style={{ color: 'green' }}>Items Ordered :</h1>
        <Cart item={CartInfo} />
      </div>
    );
  }
}
```

```

    }
  }

  export default OnlineShopping;

```

3. OUTPUT VISUALIZATION

Items Ordered :

Name	Price
Laptop	80000
TV	120000
Washing Machine	50000
Mobile	30000
Fridge	70000

Lab 3: React State Management (counterapp)

Estimated Time: 60 Minutes | Application: counterapp | Focus: Component State & Event Handlers

1. OBJECTIVES & THEORETICAL CONCEPTS

- **React State:** Built-in object used to contain data/information about the component. State changes trigger re-rendering of the UI.
- **State Updates via updater functions:** When new state depends on previous state, pass a updater function `this.setState((prevState, props) => ({ ... }))` to ensure atomic state updates.

2. SOURCE CODE IMPLEMENTATION

src/CountPeople.js

```

import React, { Component } from 'react';

class CountPeople extends Component {
  constructor() {
    super();
    this.state = {
      entrycount: 0,
      exitcount: 0,
      c: 0
    };
  }

  updateEntry = () => {
    this.setState((prevState, props) => {
      return { entrycount: prevState.entrycount + 1 };
    });
  }
}

```

```

    });

    updateExit = () => {
      this.setState((prevState, props) => {
        return { exitcount: prevState.exitcount + 1 };
      });
    };

    render() {
      return (
        <div style={{ display: 'flex', justifyContent: 'center', gap: '60px', marginTop: '40px' }}>
          <div>
            <button
              onClick={this.updateEntry}
              style={{ backgroundColor: '#90ee90', border: '1px solid #000', padding: '4px 10px',
cursor: 'pointer' }}>
              Login
            </button>
            <span style={{ marginLeft: '10px' }}>
              {this.state.entrycount} People Entered!!!
            </span>
          </div>
          <div>
            <button
              onClick={this.updateExit}
              style={{ backgroundColor: '#90ee90', border: '1px solid #000', padding: '4px 10px',
cursor: 'pointer' }}>
              Exit
            </button>
            <span style={{ marginLeft: '10px' }}>
              {this.state.exitcount} People Left!!!
            </span>
          </div>
        </div>
      );
    }
  }

  export default CountPeople;

```

3. OUTPUT VISUALIZATION

Login

3 People Entered!!!

Exit

2 People Left!!!

Lab 4: Conditional Rendering, Lists & Keys (bloggerapp)

Estimated Time: 60 Minutes | Application: bloggerapp | Focus: Array Mapping & Conditional Rendering

1. OBJECTIVES & THEORETICAL CONCEPTS

• Conditional Rendering Techniques:

- *Element Variables*: Assign JSX elements to variables based on conditions.
- *Ternary Operator* (*condition ? true : false*): Inline evaluation inside JSX.

◦ *Logical Short-Circuit (condition && element)*: Render UI only if true.

- **Keys in React**: String attributes required when rendering arrays of items to help React identify which items have changed, been added, or removed.

2. SOURCE CODE IMPLEMENTATION

src/data.js

```
export const books = [
  { id: 101, bname: 'Master React', price: 670 },
  { id: 102, bname: 'Deep Dive into Angular 11', price: 800 },
  { id: 103, bname: 'Mongo Essentials', price: 450 }
];

export const courses = [
  { id: 1, cname: 'Angular', date: '4/5/2021' },
  { id: 2, cname: 'React', date: '6/3/2021' }
];

export const blogs = [
  { id: 1, title: 'React Learning', author: 'Stephen Biz', content: 'Welcome to learning React!' },
  { id: 2, title: 'Installation', author: 'Schewzdenier', content: 'You can install React from npm.' }
];
```

src/Blogger.js

```
import React from 'react';
import { books, courses, blogs } from './data';

function Blogger() {
  // Technique 1: Element Variable
  const coursedet = (
    <ul style={{ listStyle: 'none', padding: 0 }}>
      {courses.map((course) => (
        <li key={course.id} style={{ marginBottom: '15px' }}>
          <h2 style={{ margin: '2px 0' }}>{course.cname}</h2>
          <p style={{ margin: '2px 0', fontWeight: 'bold' }}>{course.date}</p>
        </li>
      ))}
    </ul>
  );

  // Technique 2: JSX Mapping into Variable
  const bookdet = (
    <ul style={{ listStyle: 'none', padding: 0 }}>
      {books.map((book) => (
        <div key={book.id} style={{ marginBottom: '15px' }}>
          <h3 style={{ margin: '2px 0' }}>{book.bname}</h3>
          <h4 style={{ margin: '2px 0', fontWeight: 'normal' }}>{book.price}</h4>
        </div>
      ))}
    </ul>
  );

  // Technique 3: Logical Short-Circuit Content
  const content = (
    <ul style={{ listStyle: 'none', padding: 0 }}>
```



```

    {blogs.map((blog) => (
      <li key={blog.id} style={{ marginBottom: '15px' }}>
        <h2 style={{ margin: '2px 0' }}>{blog.title}</h2>
        <p style={{ fontWeight: 'bold', margin: '2px 0' }}>{blog.author}</p>
        <p style={{ color: '#555', margin: '2px 0' }}>{blog.content}</p>
      </li>
    ))}
  </ul>
);

return (
  <div style={{ display: 'table', width: '100%', textAlign: 'center' }}>
    <div style={{ display: 'table-row' }}>
      <div className="mystyle1" style={{ display: 'table-cell', width: '33%', verticalAlign:
'top', padding: '10px', borderRight: '3px solid green' }}>
        <h1>Course Details</h1>
        {coursedet}
      </div>
      <div className="st2" style={{ display: 'table-cell', width: '33%', verticalAlign: 'top',
padding: '10px', borderRight: '3px solid green' }}>
        <h1>Book Details</h1>
        {bookdet}
      </div>
      <div className="v1" style={{ display: 'table-cell', width: '33%', verticalAlign: 'top',
padding: '10px', textAlign: 'left' }}>
        <h1>Blog Details</h1>
        {content}
      </div>
    </div>
  </div>
);
}

export default Blogger;

```

3. OUTPUT VISUALIZATION

Course Details	Book Details	Blog Details
Angular	Master React	React Learning
4/5/2021	670	Stephen Biz
React	Deep Dive into Angular 11	Welcome to learning React!
6/3/20201	800	Installation
	Mongo Essentials	Schewzdenier
	450	You can install React from npm.

Lab 5: React Context API (employeeapp)

Estimated Time: 30 Minutes | Application: employeeapp | Focus: Avoiding Prop Drilling via Context

1. OBJECTIVES & THEORETICAL CONCEPTS

- **Context API:** Provides a mechanism to share global data (themes, authenticated users, locale) across component trees without explicitly passing props down through every intermediate layer ("prop drilling").
- `createContext(defaultValue)`: Creates a Context object.
- `Context.Provider`: Accepts a value prop to be consumed by descendant components.
- `useContext(Context)`: React Hook used by functional components to subscribe to context changes.

2. SOURCE CODE IMPLEMENTATION

src/ThemeContext.js

```
import { createContext } from 'react';

const ThemeContext = createContext('light');

export default ThemeContext;
```

src/App.js

```
import React, { useState } from 'react';
import ThemeContext from './ThemeContext';
import EmployeeList from './EmployeeList';

function App() {
  const [theme, setTheme] = useState('light');

  return (
    <ThemeContext.Provider value={theme}>
      <div style={{ padding: '20px' }}>
        <h1>Apps Centric Employee Management</h1>
        <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
          Toggle Theme ({theme})
        </button>
        /* Note: Theme is NOT passed as props */
        <EmployeeList />
      </div>
    </ThemeContext.Provider>
  );
}

export default App;
```

src/EmployeeList.js

```
import React from 'react';
import EmployeeCard from './EmployeeCard';

function EmployeeList() {
  const employees = [
    { id: 1, name: 'John Doe', role: 'Software Developer' }
  ];

  return (
    <div>
      {employees.map((emp) => (
```

```

        <EmployeeCard key={emp.id} employee={emp} />
      )}
    </div>
  );
}

export default EmployeeList;

```

src/EmployeeCard.js

```

import React, { useContext } from 'react';
import ThemeContext from '../ThemeContext';

function EmployeeCard({ employee }) {
  // Access context directly using useContext
  const theme = useContext(ThemeContext);

  return (
    <div style={{ border: '1px solid #ccc', margin: '10px', padding: '10px' }}>
      <h3>{employee.name}</h3>
      <p>{employee.role}</p>
      <button className={`btn-${theme}`}>
        Employee Action ({theme} mode)
      </button>
    </div>
  );
}

export default EmployeeCard;

```

Lab 6: Controlled Forms & Handlers (ticketraisingapp)

Estimated Time: 60 Minutes | Application: ticketraisingapp | Focus: Controlled Form Controls

1. OBJECTIVES & THEORETICAL CONCEPTS

- **Controlled Components:** An input form element whose value is controlled by React state. Every state mutation has an associated handler method.
- **Form Event Handling:** Capture standard submit events, prevent browser default reloads with `event.preventDefault()`, and process dynamic form values.

2. SOURCE CODE IMPLEMENTATION

src/ComplaintRegister.js

```

import React, { Component } from 'react';

class ComplaintRegister extends Component {
  constructor(props) {
    super(props);
    this.state = {
      ename: '',
      complaint: '',
      NumberHolder: 0
    };
  }
}

```


3. OUTPUT VISUALIZATION

Register your complaints here!!!

Name:

Complaint:

Lab 7: Form Validation (mailregisterapp)

Estimated Time: 60 Minutes | Application: mailregisterapp | Focus: Real-Time & Submit Validation

1. OBJECTIVES & THEORETICAL CONCEPTS

- **Validation Constraints:**

- Name: Minimum length of 5 characters.
- Email: Must contain both '@' and '.' characters.
- Password: Minimum length of 8 characters.

- **Real-time Error Tracking:** Maintain an `errors` object in component state to store active validation error messages per input field.

2. SOURCE CODE IMPLEMENTATION

src/Register.js

```
import React, { Component } from 'react';

class Register extends Component {
  constructor(props) {
    super(props);
    this.state = {
      name: '',
      email: '',
      password: '',
      errors: { name: '', email: '', password: '' }
    };
  }

  handleChange = (event) => {
    const { name, value } = event.target;
    let errors = { ...this.state.errors };

    switch (name) {
      case 'name':
```

```

        errors.name = value.length < 5 ? 'Name must be at least 5 characters!' : '';
        break;
    case 'email':
        const isValid = value.includes('@') && value.includes('.');
        errors.email = isValid ? '' : 'Email must contain @ and .!';
        break;
    case 'password':
        errors.password = value.length < 8 ? 'Password must be at least 8 characters!' : '';
        break;
    default:
        break;
}

    this.setState({ errors, [name]: value });
};

handleSubmit = (event) => {
    event.preventDefault();
    const { errors, name, email, password } = this.state;
    if (name && email && password && !errors.name && !errors.email && !errors.password) {
        alert('Registration Successful!');
    } else {
        alert('Validation Failed! Please check your entries. ');
    }
};

render() {
    const { errors } = this.state;
    return (
        <form onSubmit={this.handleSubmit} style={{ width: '300px', margin: '0 auto' }}>
            <div>
                <label>Name:</label>
                <input type="text" name="name" onChange={this.handleChange} />
                {errors.name && <div style={{ color: 'red', fontSize: '12px' }}>{errors.name}</div>}
            </div>
            <div>
                <label>Email:</label>
                <input type="text" name="email" onChange={this.handleChange} />
                {errors.email && <div style={{ color: 'red', fontSize: '12px' }}>{errors.email}</div>}
            </div>
            <div>
                <label>Password:</label>
                <input type="password" name="password" onChange={this.handleChange} />
                {errors.password && <div style={{ color: 'red', fontSize: '12px' }}>{errors.password}</div>}
            </div>
            <button type="submit">Register</button>
        </form>
    );
}
}

export default Register;

```

Lab 8: Consuming REST APIs (fetchuserapp)

Estimated Time: 60 Minutes | Application: fetchuserapp | Focus: Asynchronous HTTP Fetching

1. OBJECTIVES & THEORETICAL CONCEPTS

- **REST API Integration:** Fetch external data over HTTP using standard modern JavaScript Promises or `async/await` syntax.
- **Lifecycle Hook Integration:** Invoke asynchronous network requests inside `componentDidMount()` lifecycle method to ensure data fetching occurs post-mounting.

2. SOURCE CODE IMPLEMENTATION

src/Getuser.js

```
import React, { Component } from 'react';

class Getuser extends Component {
  constructor(props) {
    super(props);
    this.state = {
      person: null,
      loading: true
    };
  }

  async componentDidMount() {
    const url = "https://api.randomuser.me/";
    const response = await fetch(url);
    const data = await response.json();
    this.setState({ person: data.results[0], loading: false });
  }

  render() {
    return (
      <div style={{ textAlign: 'center', marginTop: '30px' }}>
        {this.state.loading || !this.state.person ? (
          <div>loading...</div>
        ) : (
          <div>
            <h3>
              {this.state.person.name.title}. {this.state.person.name.first}
            </h3>
            <img src={this.state.person.picture.large} alt="User" />
          </div>
        )}
      </div>
    );
  }
}

export default Getuser;
```

3. OUTPUT VISUALIZATION

Mr. Brad Gibson

[User Image]

